

MAKERSPACE WORKSHOP

AI for Makers

From prompt to prototype

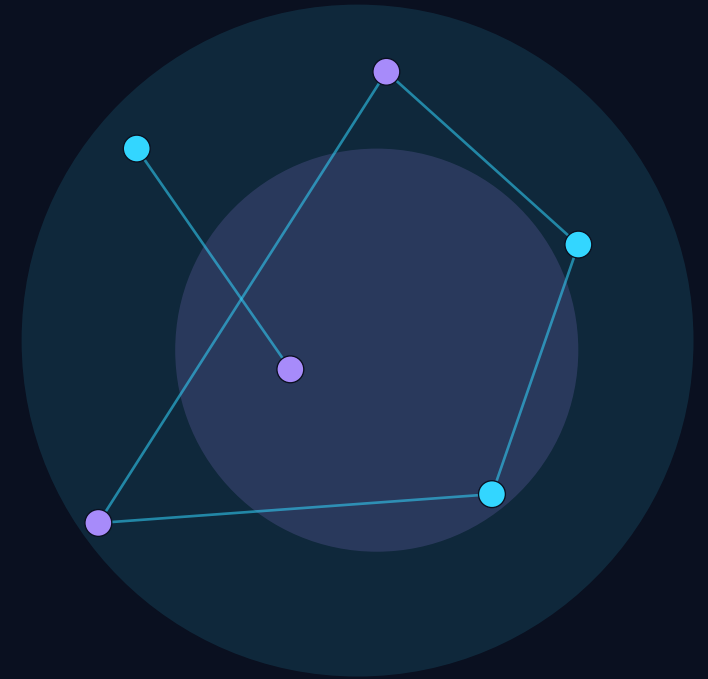
How LLMs work • Tokens • Prompting • Troubleshooting • Coding • Research
• Model selection • Local & air-gapped AI • Safety

~55 MIN + Q&A

LIVE DEMOS

SOURCE-BACKED

Prepared for a mixed technical + beginner audience



What you should be able to do by the end

A practical mental model, not a vocabulary test.

1

Explain what an LLM is

...and why it can sound confident while being wrong.

2

Write better prompts

Use a repeatable structure instead of “magic words.”

3

Choose the right tool

Match task → product → model → tools → privacy level.

4

Run AI locally

Understand Ollama, Open WebUI, model sizes, quantization, and air gaps.

5

Verify important answers

Use sources, tests, logs, measurements, and independent benchmarks.

AI is not one thing

Separate the interface from the model and the model from the tools.

PRODUCT / UI

ChatGPT • Claude • Gemini • Perplexity • Copilot

AGENT / WORKFLOW

Hermes • OpenClaw • coding agents • automations

ROUTER / GATEWAY

OpenRouter • model gateways • fallback / cost routing

MODEL

GPT • Claude • Gemini • Qwen • Gemma • Llama

RUNTIME / HARDWARE

Ollama • llama.cpp • GPU / CPU / RAM

Why this matters: “I use ChatGPT” does not tell you which model, tools, privacy mode, or data path produced the answer.

[Sources: OpenRouter docs](#)

[• Open WebUI docs](#)

[• Hermes docs](#)

[• OpenClaw docs](#)

How an LLM works — useful simplification

Text goes in. The model repeatedly predicts a next token based on the context it can see.

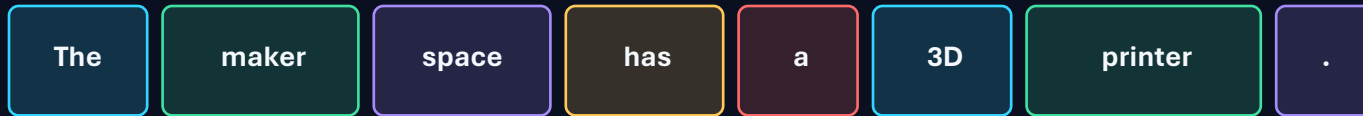


Important: “predicting the next token” does not mean “simple autocomplete.” The learned transformer has billions of parameters and can encode rich statistical structure — but generation is still probabilistic, not a database lookup.

Tokens: the unit the model actually sees

A token can be a word, part of a word, punctuation, whitespace, or another learned text chunk.

EXAMPLE — CONCEPTUAL ONLY



Tokens affect cost

Cloud APIs commonly meter input and output in tokens. More context and longer answers generally mean more usage.

Tokens affect capacity

A context window is measured in tokens. Once the usable window is full, content must be dropped, summarized, retrieved, or otherwise managed.

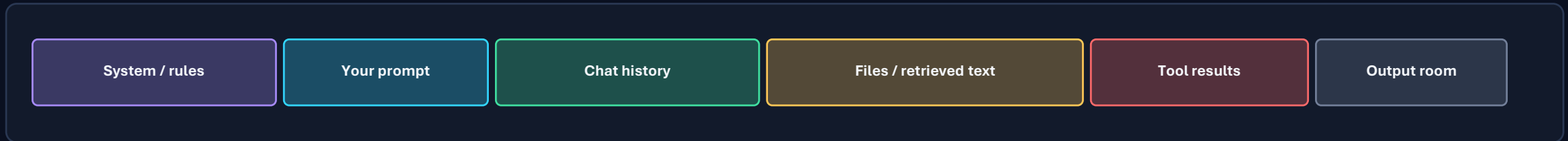
Tokens ≠ words

Tokenization differs by model and language. Never assume “1 token = 1 word.” Use the model’s tokenizer or token-count API.

OpenAI’s rule of thumb for English: ~1 token ≈ 4 characters or ~¾ of a word — useful for estimating, not for exact counts.

Context is working memory — not perfect memory

The model can only act on information that reaches its current context or tools.



Longer is not always better

Huge context can increase cost and make relevant information harder to locate. Curate the evidence you actually need.

RAG retrieves on demand

Retrieval-augmented generation searches your documents or knowledge base and inserts relevant chunks into context.

“Memory” is a product feature

Saved memory, project context, vector stores, and chat history are surrounding systems—not a magical permanent memory inside one model response.

Maker tip: give the model the exact manual page, error log, schematic photo, or code file you are discussing instead of asking it to guess.

Why AI hallucinates — and what to do about it

Fluent language is not proof. Confidence and correctness are separate variables.

WHY ERRORS HAPPEN

- The model is optimized to produce plausible continuations
- Its training data may be incomplete, stale, or conflicting
- A prompt may omit key facts or contain bad assumptions
- Tools/search can return poor sources or be unavailable

HOW TO REDUCE RISK

- Provide primary evidence: logs, manuals, measurements, code
- Ask for assumptions, uncertainty, and missing information
- Use web/search or retrieval for fresh / obscure facts
- Test the answer: run code, measure voltage, reproduce the bug
- For high-stakes decisions, verify with qualified sources

Rule: AI output is a hypothesis until the evidence or test makes it a result.

Prompting: give the model a job specification

Clear context and constraints beat incantations. Prompting is iterative engineering.

GOAL

What outcome do you want?

CONTEXT

What does it need to know?

CONSTRAINTS

Limits, safety, tools, budget, environment

EVIDENCE

Logs, files, photos, docs, URLs, measurements

OUTPUT

Format, depth, audience, checklist/code/table

ITERATE

Test → correct → provide new evidence

Example: “You are a senior 3D Printing Engineer. I have a Prusa-style printer. PETG is stringing after a nozzle change. Here are my slicer settings and a photo. Give me the three most likely causes in priority order, one safe test for each, and tell me what evidence would rule each cause out.”

Example: “You are a Principal Prompt Engineer. I have a 3D printer question that I need to create an AI prompt for to help me solve my problem...(insert problem here). Ask me clarifying questions and QA with me until we have a ready to use prompt.”

Source: [Google: Prompt design strategies](#)

Use case: troubleshooting a physical or technical problem

AI is strongest when you make it follow a diagnostic loop instead of jumping to a fix.



Good maker inputs

Error codes • serial logs • photos • wiring diagram • exact model number • firmware/software version • measurements • “what changed?”

Tell AI to avoid

Replacing five parts at once • unsafe electrical guesses • bypassing protections • assuming a photo proves hidden damage • claiming certainty without a test

Use case: code development & debugging

Treat AI like a fast pair programmer whose work still needs tests and review.

Great for

- Explaining unfamiliar code
- Generating scaffolding & tests
- Refactoring with constraints
- Interpreting stack traces
- Drafting regex / SQL / scripts

Give it

- Relevant files / snippets
- Exact error + reproduction
- Language & version
- Expected vs actual behavior
- Tests / lint / compiler output

Do not trust blindly

- Dependency versions
- Security-sensitive code
- Destructive shell commands
- Concurrency / data loss edge cases
- “It compiles” without running it

Best loop: ask → diff → run tests → inspect failures → feed results back → repeat.

Source: [GitHub Copilot: models differ by task, latency, and hallucination behavior](#)

Use case: investigation, search, and research

The model is the synthesis layer. The sources are the evidence layer.



Research prompt pattern *“Search the current web. Prefer primary sources. For every material claim, cite the source. Include publication date vs event date. Identify disagreements, missing evidence, and what would make the conclusion change.”*

A citation is not proof until you click it.

Maker-space use cases: where AI can multiply your iteration speed

Use AI to reduce lookup and drafting time — then validate against the physical world.

3D PRINTING

Slicer settings • failure diagnosis • G-code explanation • material comparisons

ELECTRONICS

Datasheet extraction • resistor / power math • Arduino / ESP code • serial-log diagnosis

CAD / FAB

Parametric design ideas • tolerance checklists • CAM setup questions • documentation

WOOD / METAL

Cut lists • jig ideas • process checklists • tool/manual lookup — with safety verification

CRAFT / TEXTILE

Pattern adaptation • material calculations • instructions • visual ideation

HOMELAB / IOT

Docker • Linux • networking • Home Assistant • logs • scripts • config review

DOCUMENTS

Manual summarization • comparison tables • BOMs • labels • class handouts

LEARNING

Explain at multiple skill levels • quizzes • vocabulary • guided practice

High leverage pattern: photograph / attach the artifact + identify the exact model/version + ask for a testable next step.

Choosing a cloud AI product: start with the workflow

There is no universal “best AI.” Pick the ecosystem and capabilities that fit the job.

ChatGPT / OpenAI	Broad general work, coding, research, multimodal creation, tool-rich workflows	Strong all-around product; current GPT-5.6 family spans flagship → lower-cost tiers
Claude / Anthropic	Long-context work, coding, writing, agentic enterprise tasks	Current family explicitly tiers Fable / Opus / Sonnet / Haiku by capability and latency
Gemini / Google	Google ecosystem, multimodal, coding/agentic work, media models	Multiple Pro/Flash/Live/media variants; model catalog changes quickly
GitHub Copilot	Software development inside IDE/GitHub workflows	Model picker + Auto; models differ in latency, cost, hallucination profile, and task fit
Perplexity	Web-first investigation and source-backed synthesis	Search modes + multiple models; product centers citations and web retrieval

Choose on: task quality • fresh-data access • tool integrations • privacy • latency • cost • context • team workflow — not brand loyalty.

Sources: [OpenAI](#) • [Anthropic](#) • [Google](#) • [GitHub](#) • [Perplexity](#)

Model families snapshot — August 16, 2026

Names change. Learn the pattern: flagship / balanced / fast + specialist models.

OPENAI

- GPT-5.6 Sol — flagship
- GPT-5.6 Terra — balanced
- GPT-5.6 Luna — fastest / lowest-cost tier

ANTHROPIC

- Claude Fable 5 — highest broadly released capability
- Claude Opus 5 — complex agentic / enterprise
- Claude Sonnet 5 — speed + intelligence
- Claude Haiku 4.5 — fastest

GOOGLE

- Gemini 3.1 Pro — advanced problem solving (preview)
- Gemini 3.7 Flash — latest capable Flash
- Flash-Lite — high-throughput / lower cost
- Specialized Live / image / video models

LOCAL / OPEN WEIGHTS

- Qwen families
- Gemma families
- Llama families
- Many sizes + quantizations

Model menus are live inventory. Before a class, project, or purchase decision, check the vendor's current model page and release notes.

[Sources: OpenAI current family](#)

[• Claude current family](#)

[• Gemini current catalog](#)

[• Ollama model library](#)

How to read AI benchmarks without getting fooled

A leaderboard is evidence about a test — not a universal answer to “which model is best?”

1 What does it test? Coding? preference? math? agent tools? long context?

2 Who ran it? Vendor self-report or independent evaluator?

3 What harness? Prompting, tools, reasoning effort, number of attempts, temperature?

4 What version/date? Models and benchmarks both change.

5 What tradeoff? Quality vs latency vs cost vs tokens vs hardware?

USEFUL PLACES TO LOOK

Artificial Analysis

Independent composite + speed / cost methodology

[open](#)

Arena

Blind human preference comparisons + public policy

[open](#)

LiveBench

Frequently refreshed objective benchmark set

[open](#)

SWE-bench

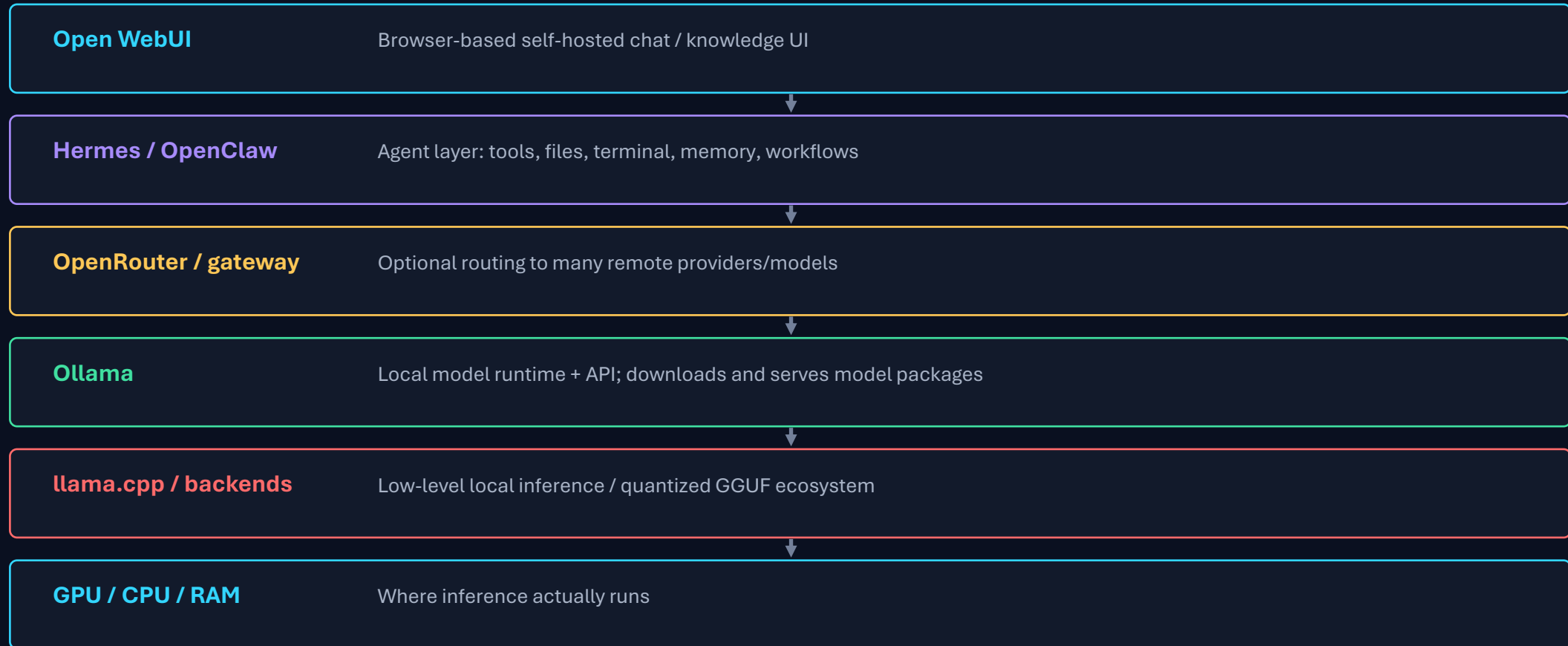
Software engineering benchmark ecosystem

[open](#)

Benchmark caution: even test suites can contain flawed cases.

Local AI stack: what each component does

“Local” can mean local UI, local model, local storage, or fully isolated network. Be specific.



Cloud path can still exist inside a “local” stack. Follow the arrows and ask: where does the prompt leave my machine?

Sources: Ollama

• Open WebUI

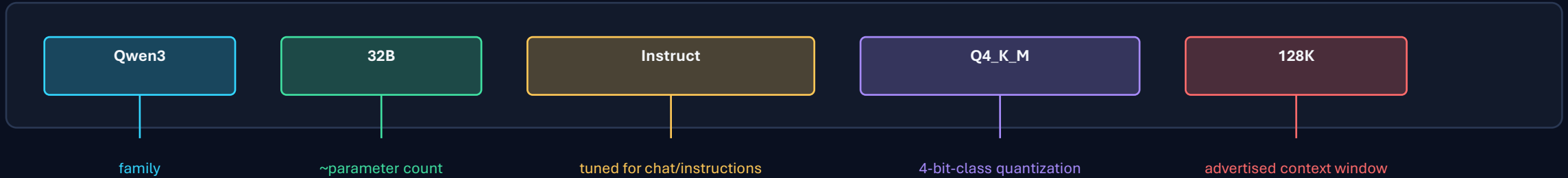
• OpenRouter

• Hermes

• OpenClaw

Reading a local model name: 31B, Q4, 128K, Instruct...

The label tells you different things about architecture, precision, context, and tuning.



“B” = billion parameters

32B means roughly 32 billion learned parameters. Parameter count is not the same as file size, RAM use, or quality.

Quantization shrinks weights

llama.cpp supports low-bit quantization (including 4-bit classes), reducing model size / memory at some potential quality cost.

Context also consumes memory

Longer context can require substantial additional runtime memory; “128K supported” does not mean 128K is always practical on your hardware.

Rule of thumb for weights only: 32B at ~4 bits ≈ 16 GB before format/runtime overhead. Real usage is higher.

Local model sizing: practical starting points

Approximate guidance for quantized models. Exact needs vary with architecture, quantization, context, GPU offload, and concurrency.

Model tier	Typical Q4-ish package	Practical RAM	Comfortable GPU VRAM	Good for
1–4B	~1–4 GB	8 GB RAM	4–6 GB VRAM	Fast helpers, extraction, simple chat
7–9B	~5–7 GB	16 GB RAM	8 GB VRAM	Excellent maker-space starting tier
12–14B	~8–10 GB	24 GB RAM	12–16 GB VRAM	Stronger reasoning / coding
27–32B	~17–22 GB	32–48 GB RAM	24 GB VRAM	High-quality single-GPU target
70B	~40+ GB	64+ GB RAM	48+ GB VRAM*	High-end / multi-GPU / CPU-offload territory

*“Comfortable GPU VRAM” is workshop guidance, not a vendor minimum. CPU/RAM offload can run larger models more slowly. Context/KV cache and multiple users add memory.

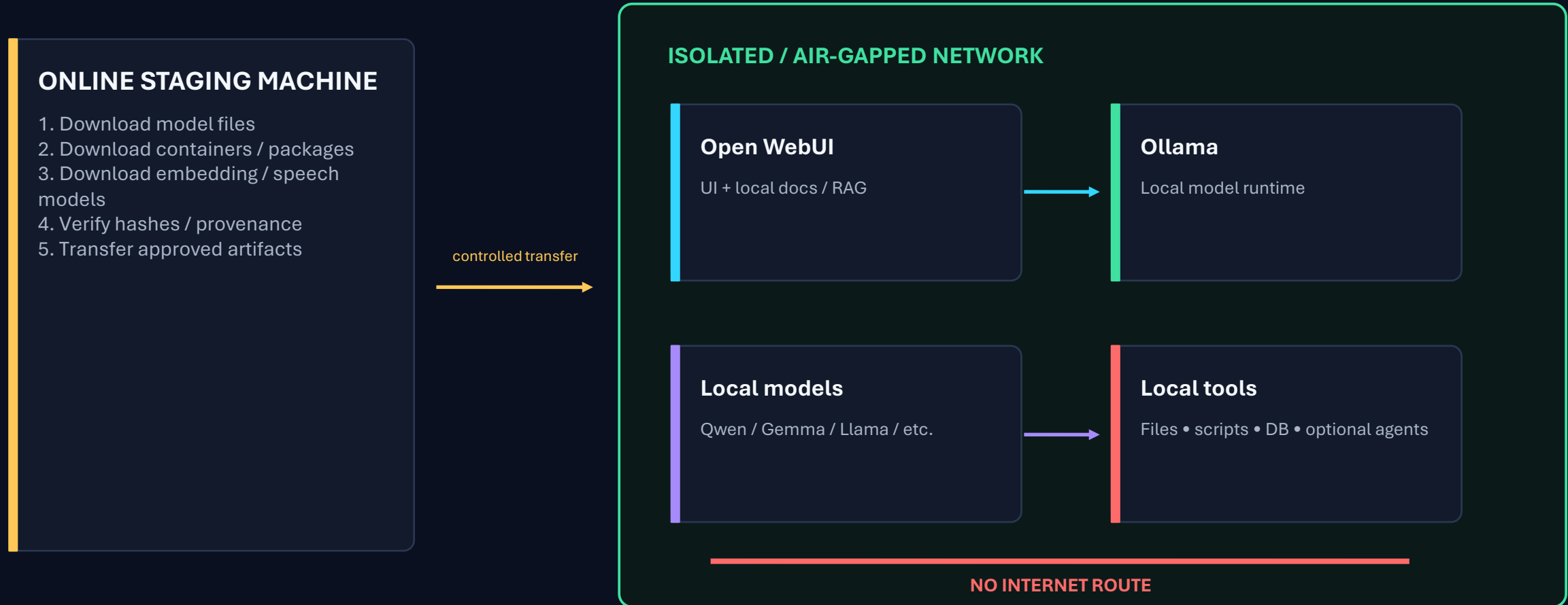
Concrete proof point: Ollama lists Gemma 3 packages around 3.3 GB (4B), 8.1 GB (12B), and 17 GB (27B); the same family supports quantization-aware variants with lower memory footprint.

Sources: Ollama Gemma 3 package sizes

• llama.cpp quantization

Air-gapped AI: what “offline” actually requires

Software offline mode is helpful. A true air gap is enforced by network isolation.

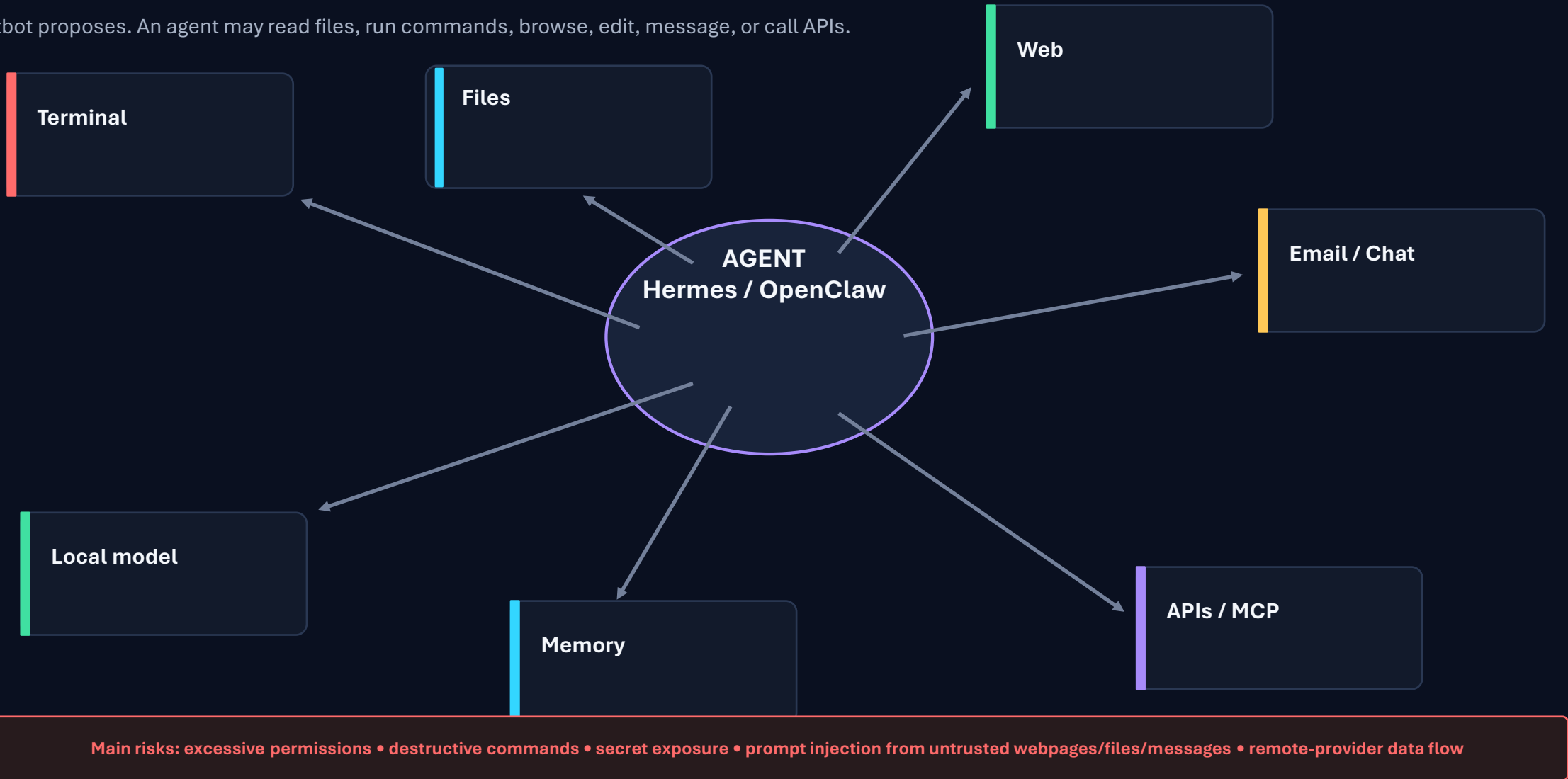


Open WebUI’s docs explicitly say full air-gapping requires isolating the instance from the internet; OFFLINE_MODE alone is not the network boundary.

Source: [Open WebUI Offline Mode](#)

Agents are different: they can take actions

A chatbot proposes. An agent may read files, run commands, browse, edit, message, or call APIs.



Sources: [Hermes Agent docs](#)

• [OpenClaw docs / prompt-injection warning](#)

What is safe to paste into AI? Use a data-classification rule

Do not decide based on “AI good / AI bad.” Decide based on sensitivity + provider + configuration + tools.

Class	Examples	Where to use	Rule
PUBLIC	Published manuals, public code, public product specs	Cloud or local	Still verify accuracy
INTERNAL	Non-public project notes, draft designs, ordinary logs	Approved business service or controlled local	Check organization policy and retention settings
CONFIDENTIAL	Client data, proprietary source, private documents	Only explicitly approved environment	Minimize, redact, access-control, log
SECRET / REGULATED	Passwords, API keys, private keys, sensitive personal/regulated data	Do not paste casually; use dedicated approved controls	Prefer secrets managers / specialized workflows; local alone is not sufficient

“Self-hosted” does not automatically mean “private”: plugins, agents, remote APIs, browser tools, telemetry, and update checks can create outbound paths.

Sources: [Open WebUI hardening](#)

• [OpenClaw data-flow caveats](#)

• [OpenAI API data controls](#)

Live demo plan — 12 to 15 minutes

Three demos that teach transferable behavior instead of brand-specific tricks.

DEMO 1 — PROMPT UPGRADE

Start vague → add evidence + constraints → compare the answer
Example: “My 3D print is bad” → attach photo/settings and ask for ranked tests.

4 min

DEMO 2 — VERIFY A CLAIM

Ask a current technical question → require sources → open one source
Show that a polished answer still needs source verification.

4 min

DEMO 3 — LOCAL VS CLOUD

Run the same small task in a local model and a frontier cloud model
Compare latency, privacy path, depth, and hardware—not just wording.

4–6 min

If Wi-Fi fails: demos 1 and 3 can still work entirely locally if the model and UI are already downloaded.

Copy/paste demo prompts

These are designed to expose uncertainty and produce testable next steps.

Troubleshooting

I am diagnosing [device/problem]. Symptoms: [symptoms]. Environment/version: [details]. Evidence: [logs/photo/settings]. Rank the 3 most likely causes. For each: explain why, give ONE safe test that distinguishes it, and say what observation would rule it out. Do not recommend replacing parts until the evidence supports it.

Code debugging

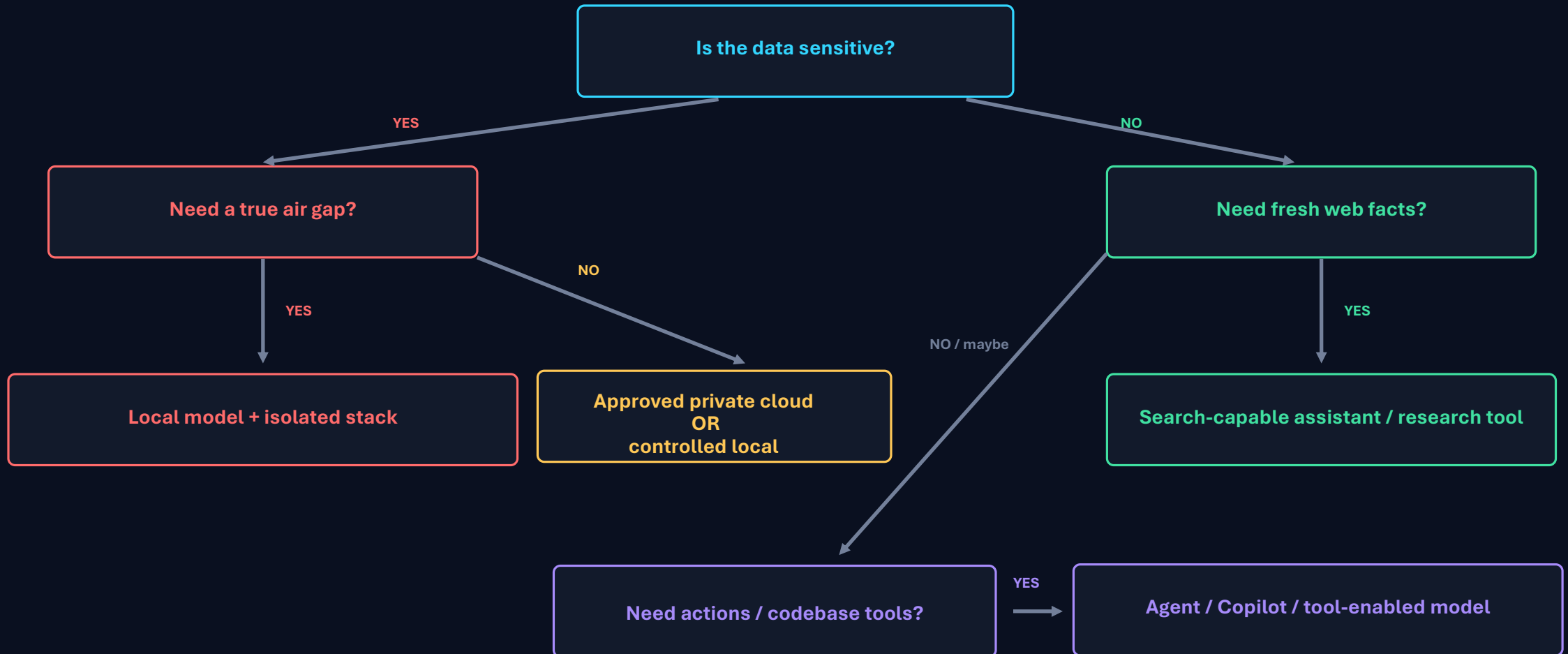
Review the attached code and error. First reproduce the failure mentally from the evidence. Then identify the smallest likely root cause. Propose a minimal patch, explain the risk, and give me tests that would fail before the patch and pass after it. If a dependency/version fact is uncertain, say so and tell me what command/doc to check.

Research / purchasing

Research [question] using current sources. Prefer manufacturer/official documentation for specifications and independent sources for comparative claims. Cite every material claim. Separate verified facts from inference. Include dates. End with: “What could make this recommendation wrong?”

Which AI should I use? A 30-second decision tree

Start with data sensitivity and whether the task needs fresh information or actions.



Final tie-breakers: quality on YOUR task → privacy → latency → cost → hardware → workflow fit.

The five rules to remember tomorrow

If the audience remembers only this slide, the workshop succeeded.

1 AI generates; evidence verifies.

Treat outputs as hypotheses until tested or sourced.

2 Give context, constraints, and artifacts.

Photos, logs, code, manuals, measurements beat vague prompts.

3 Choose for the task, not the logo.

Models trade intelligence, speed, cost, tools, context, and privacy.

4 Local ≠ automatically private.

Trace network paths, tools, plugins, permissions, and provider settings.

5 Agents deserve least privilege.

The ability to act creates a different risk class than chat.

Source pack — primary docs + independent benchmarks

Clickable links. Model pages are intentionally live because the market changes fast.

How transformers work	Attention Is All You Need	Human preference	Arena leaderboard policy
Tokens / tokenizer	OpenAI token guide + tokenizer	Fresh benchmark	LiveBench
OpenAI models	GPT-5.6 family / evaluations	Coding benchmark	SWE-bench
Anthropic models	Claude model overview	Local runtime	Ollama
Google models	Gemini API model catalog	Quantization	llama.cpp quantize docs
Copilot models	GitHub model comparison	Offline / air gap	Open WebUI offline mode
Perplexity	Search + model documentation	Agent framework	Hermes Agent docs
Independent evals	Artificial Analysis methodology	Agent framework	OpenClaw docs / FAQ

Tip: for a future version of this deck, re-check the vendor model pages and the benchmark dates first.

What are you building?

Bring a real project, error, model name, or “can AI do this?” question.

Good Q&A targets

Prompt review • model choice • local hardware •
privacy path • debugging workflow

Bring evidence

Screenshot • log • code • manual • photo • model
card • benchmark

We can compare

Cloud vs local • small vs large • search vs no-
search • agent vs chat

The best AI workflow is the one you can verify.